

# Lecture 7 — CPU Hardware, Branch Prediction

Patrick Lam & Jeff Zarnett

`patrick.lam@uwaterloo.ca jzarnett@uwaterloo.ca`

Department of Electrical and Computer Engineering  
University of Waterloo

September 27, 2026



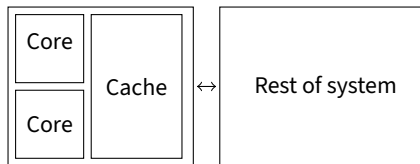
Turning up the clock rate makes chips get too hot.

Now there is base rate ( $\sim 3\text{GHz}$ ) and burst speeds;  
sensors check that the chip isn't too hot for too long while bursting.

Identical processors or cores, which:

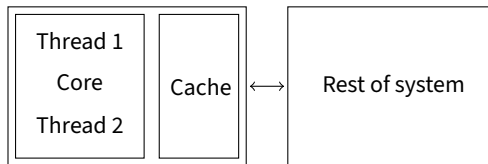
- are interconnected, usually using buses; and
  - share main memory.
- 
- SMP is most common type of multiprocessing system.

# Example of an SMP System



- Each core can execute a different thread
- Shared memory quickly becomes the bottleneck

# Executing 2 Threads on a Single Core



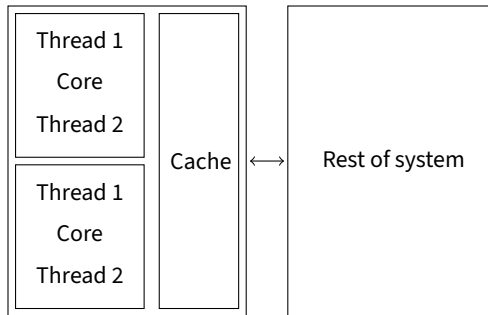
On a single core, must context switch between threads:

- every  $N$  cycles; or
- wait until cache miss, or another long event

Resources may be unused during execution.

Why not take advantage of this?

# Executing M Threads on a N Cores



Here's a Chip Multithreading example.

UltraSPARC T2 has 8 cores, each of which supports 8 threads. All of the cores share a common level 2 cache.

AMD Ryzen 5 (Zen 4) has 6 cores and 12 threads.  
L1, L2 per-core; L3 shared per-CCD.

# SMT (Simultaneous Multithreading)

Use idle CPU resources (may be calculating or waiting for memory) to execute another task.

Cannot improve performance if shared resources are the bottleneck.

Issue instructions for each thread per cycle.

To the OS, it looks a lot like SMP, but gives only up to 30% performance improvement.

Intel implementation: Hyper-Threading.

## Example: Non-SMP system



PlayStation 3 contains a Cell processor:

- PowerPC main core (Power Processing Element, or “PPE”)
- 7 Synergistic Processing Elements (“SPE”s): small vector computers.

Intel: P (performance) and E (efficient cores).

E cores are higher-latency, but smaller (more can fit).

P cores are lower-latency.

AMD: c-cores similar to E cores.

The programming interfaces for these cores are uniform on the same CPU.

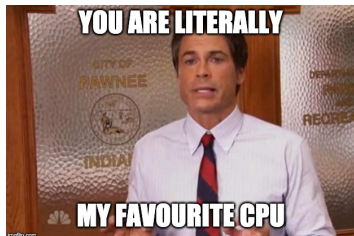
# NUMA (Non-Uniform Memory Access)

In SMP, all CPUs have uniform (the same) access time for resources.

For NUMA, CPUs can access different resources faster  
(resources: not just memory).

Schedule tasks on CPUs which access resources faster.

Since memory is commonly the bottleneck,  
each CPU has its own memory bank.



Each task (process/thread) can be associated with a set of processors.

Useful to take advantage of existing caches (either from the last time the task ran or task uses the same data).

Hyper-Threading is an example of complete affinity for both threads on the same core.

Often better to use a different processor if current set busy.

The compiler (& CPU) take a look at code that results in branch instructions.

Examples: loops, conditionals, or the dreaded goto.

It will take an assessment of what it thinks is likely to happen.

In the beginning the CPUs/compiler didn't really think about this sort of thing.

They come across instructions one at a time and do them and that was that.

If one of them required a branch, it was no real issue.

Then we had pipelining...

The CPU would fetch the next instruction while decoding the previous one, and while executing the instruction before.

That means if evaluation of an instruction results in a branch, we might go somewhere else and therefore throw away the contents of the pipeline.

Thus we'd have wasted some time and effort.

If the pipeline is short, this is not very expensive.  
But pipelines keep getting longer...

CPU: \*predicts wrong execution branch\*



The compiler and CPU look at instructions on their way to be executed and analyze whether it thinks it's likely the branch is taken.

This can be based on several things, including the recent execution history.

If we guess correctly, this is great, because it minimizes the cost of the branch.

If we guess wrong, we flush the pipeline and take the performance penalty.

The compiler and CPU's branch prediction routines are pretty smart.  
Trying to outsmart them isn't necessarily a good idea.

But we can give the compiler some hints  
about what we think is likely to happen.



We can either say that something is likely or unlikely.

This is an experimental feature in Rust only available in nightly builds.

# What is the effect of hints?

---

```
fn f(a: i32) -> i32 {
    a
}

fn main() {
    let size = 100000;
    let large_vector = vec![0; size];
    let mut m1 = 0;
    let mut m2 = 0;

    for _j in 0..1000 {
        for k in 0..size {
            if *large_vector.get(k).unwrap() == 0 {
                m1 = f(m1 + 1)
            } else {
                m2 = f(m2 + 1)
            }
        }
    }
    println!("m1={}, m2={}", m1, m2);
}
```

---

I'll first test it with no hint, then putting `likely()` around the if condition, and then `unlikely()`.

Running it yielded:

No hint at all:

|                      |                     |                                  |
|----------------------|---------------------|----------------------------------|
| Time (mean +/- ?):   | 6.657 s +/- 0.144 s | [User: 6.614 s, System: 0.029 s] |
| Range (min ... max): | 6.413 s ... 6.905 s | 10 runs                          |

Likely:

|                      |                     |                                  |
|----------------------|---------------------|----------------------------------|
| Time (mean +/- ?):   | 6.762 s +/- 0.175 s | [User: 6.729 s, System: 0.028 s] |
| Range (min ... max): | 6.590 s ... 7.200 s | 10 runs                          |

Unlikely:

|                      |                     |                                  |
|----------------------|---------------------|----------------------------------|
| Time (mean +/- ?):   | 6.943 s +/- 0.200 s | [User: 6.893 s, System: 0.033 s] |
| Range (min ... max): | 6.732 s ... 7.309 s | 10 runs                          |

Hints don't help the compiler in this program!

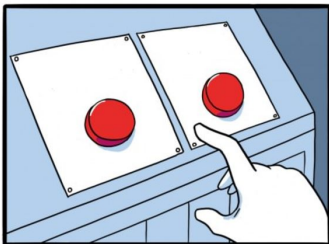
We proved they aren't always a benefit, even if we know they are correct.

Conclusion: it's hard to outsmart the compiler. Maybe it's better not to try.

I want you to pick up two points from this next discussion:

- How branch predictors work
- Applying a (straightforward) expected value computation to predict performance.

# Branch Prediction & Misprediction



JAKE-CLARK.TUMBLR

---

```
branch_if_not_equal x, 0,  
    else_label  
// Do stuff  
goto end_label  
else_label:  
// Do things  
end_label:  
// whatever happens later
```

---

With no prediction, we need to serialize:

|       |       |          |          |
|-------|-------|----------|----------|
| bne.1 | bne.2 |          |          |
|       |       | things.1 | things.2 |

If our prediction is correct, we save time.

|       |          |          |
|-------|----------|----------|
| bne.1 | bne.2    |          |
|       | things.1 | things.2 |

But we might be wrong and need to throw out the bad prediction.

|       |          |         |         |
|-------|----------|---------|---------|
| bne.1 | bne.2    |         |         |
|       | things.1 |         |         |
|       |          | stuff.1 | stuff.2 |

We need to quantify the performance.

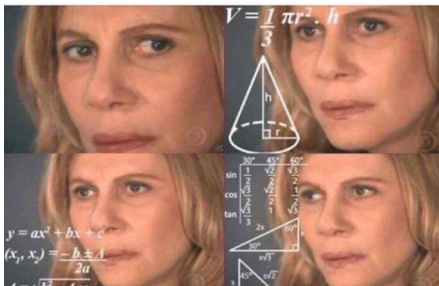
Let's pretend that our pipelined CPU executes, on average, one instruction per clock.

Mispredicted branches cost 20 cycles, while correctly-predicted branches cost 1 cycle.

We'll also assume that the instruction mix contains 80% non-branches and 20% branches.

So we can predict average cycles per instruction.

# Prediction Quantification



With no prediction (or always-wrong prediction):

$$\text{non\_branch\_}\% \times 1 \text{ cycle} + \text{branch\_}\% \times 20 \text{ cycles} = 4.8 \text{ cycles.}$$

With perfect branch prediction:

$$\text{non\_branch\_}\% \times 1 \text{ cycle} + \text{branch\_}\% \times 1 \text{ cycle} = 1 \text{ cycle.}$$

So we can make our code run  $4.8\times$  faster with branch prediction!

We can predict that a branch is always taken.

If we got 70% accuracy, then our cycles per instruction would be:

$$(0.8 + 0.7 \times 0.2) \times 1 \text{ cycle} + (0.3 \times 0.2) \times 20 \text{ cycles} = 2.14 \text{ cycles}.$$

The simplest possible thing already greatly improves the CPU's average throughput.

Let's leverage that observation about loop branches to do better. Loop branches are, by definition, backwards.

So we can design a branch predictor which predicts “taken” for backwards and “not taken” for forwards.

Let's say that this might get us to 80% accuracy.

$$(0.8 + 0.8 \times 0.2) \times 1 \text{ cycle} + (0.2 \times 0.2) \times 20 \text{ cycles} = 1.76 \text{ cycles.}$$

The PPC 601 (1993) and 603 used this scheme.

So far, we will always make the same prediction at each branch – known as a **static** scheme.

But we can do better by using what recently happened to improve our predictions.

This is particularly important when program execution contains distinct phases, with distinct behaviours.

We therefore move to **dynamic** schemes.

For every branch, we record whether it was taken or not last time it executed (a 1-bit scheme).

Of course, we can't store all branches.

So let's use the low 6 bits of the address to identify branches.

Doing so raises the prospect of *aliasing*: different branches (with different behaviour) map to the same spot in the table.

We might get 85% accuracy with such a scheme.

$$(0.8 + 0.85 \times 0.2) \times 1 \text{ cycle} + (0.15 \times 0.2) \times 20 \text{ cycles} = 1.57 \text{ cycles.}$$

At the cost of more hardware, we get noticeable performance improvements. The DEC EV4 (1992) and MIPS R8000 (1994) used this one-bit scheme.

What if a branch is almost always taken but occasionally not taken (e.g., TTTTTTNTTTT)?

We get penalized twice for that misprediction: once when we mispredict the not taken, and once when we mispredict the next taken.

So, let's store whether a branch is “usually” taken, using a so-called 2-bit saturating counter.

Every time we see a taken branch, we increment the counter for that branch; every time we see a not-taken branch, we decrement.

If the counter is 00 or 01, we predict “not taken”; if it is 10 or 11, we predict “taken”.

With a two-bit counter, we can have fewer entries at the same size, but they'll do better. It would be reasonable to expect 90% accuracy.

$$(0.8 + 0.9 \times 0.2) \times 1 \text{ cycle} + (0.1 \times 0.2) \times 20 \text{ cycles} = 1.38 \text{ cycles.}$$

This was used in a number of chips, from the LLNL S-1 (1977) through the Intel Pentium (1993).

We're still not taking patterns into account. Consider the following for loop.

---

```
for (int i = 0; i < 3; ++i) {  
    // code  
}
```

---

The last three executions of the branch determine the next direction:

TTT => N

TTN => T

TNT => T

NTT => T

Let's store what happened the last few times we were at a particular address – the **branch history**.

From a branch address and history, we derive an index, which points to a table of 2-bit saturating counters.

What's changed from the two-bit scheme is that the history helps determine the index and hence the prediction.

This scheme might give something like 93% accuracy.

$$(0.8 + 0.93 \times 0.2) \times 1 \text{ cycle} + (0.07 \times 0.2) \times 20 \text{ cycles} = 1.27 \text{ cycles.}$$

The Pentium MMX (1996) used a 4-bit global branch history.

The change here is that the CPU keeps a separate history for each branch.

So the branch address determines which branch history gets used.

We concatenate the address and history to get the index, which then points to a 2-bit counter again.

We are starting to encounter diminishing returns, but we might get 94% accuracy:

$$(0.8 + 0.94 \times 0.2) \times 1 \text{ cycle} + (0.06 \times 0.2) \times 20 \text{ cycles} = 1.23 \text{ cycles.}$$

The Pentium Pro (1996), Pentium II (1997) and Pentium III (1999) use this.

Instead of concatenating the address and history, we can xor them.

This allows us to use more bits for both the history and address.

This keeps the accuracy the same, but simplifies the design.

We can build (and people have built) more sophisticated predictors.

These predictors could, for instance, better handle aliasing, where different branches/histories map to the same index in the table.

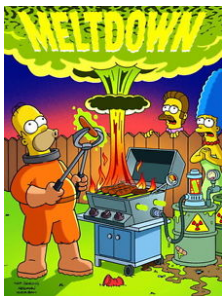
# Hacking Time... Hacking Too Much Time



A few years ago, a lot happened in terms of exploiting the hardware of CPU architectures to get access to privileged data.

Unfortunately these things have performance implications!

# Cache side-channel attacks





These attacks leverage performance features of modern CPUs to break process isolation guarantees.

In principle, a process shouldn't be able to read memory that belongs to the kernel or to other processes.

Spectre and Meltdown can cause privileged memory to be loaded into the cache, and then extracted using a cache side-channel attack.

Mitigation: more isolation, lower performance.

# Vulnerability Example

```
struct array {
    unsigned long length;
    unsigned char data[];
};

struct array *arr1 = ...; /* small array */
struct array *arr2 = ...; /* array of size 0x400 */
/* >0x400 (OUT OF BOUNDS!) */
unsigned long untrusted_offset_from_caller = ...;
if (untrusted_offset_from_caller < arr1->length) {
    unsigned char value = arr1->data[untrusted_offset_from_caller];
    unsigned long index2 = ((value&1)*0x100)+0x200;
    if (index2 < arr2->length) {
        unsigned char value2 = arr2->data[index2];
    }
}
```

(from <https://hk.saowen.com/a/81873c7b149c0993836e8a4fa4f879b4178085a3ad14c09a5f22f5a9c76373ca>)

Remember that in hyperthreading, two threads are sharing the same execution core. That means they have hardware in common.

Because of this, a thread can figure out what the other thread is doing!

By noticing its cache accesses and by timing how long it takes to complete operations.

Attack name: PortSmash

In the practical example, a 384-bit secret key is (over time) completely stolen by another process.

Mitigation: prevent threads from different processes from using the same core...

Possibly the only long term solution is to not use hyperthreading at all...

The performance implications of which are obvious & significant!